Spec Driven Development (SDD)

A Practical Guide for HTML & Frontend Developers

Introduction - What is Spec Driven Development (SDD)

Spec Driven Development (SDD) is a development approach where we **write clear specifications before writing code**.

Instead of directly jumping into coding, we first define:

- What we want to build
- How it should behave
- What edge cases exist
- How components interact

Once everything is clearly defined in **specs**, AI tools and developers can generate or implement code **more accurately and consistently**.

Traditional Development:
Idea → Code → Debug → Fix → Refactor → Confusion

Spec Driven Development
Idea → Specification → Plan → Tasks → Implementation

Why SDD is Useful

SDD helps solve common development problems:

Problem	Solution with SDD
unclear requirements	specs define them clearly
inconsistent implementations	specs guide developers
miscommunication between teams	specs act as a shared contract
AI hallucination in coding	specs give AI strict instructions

Simple Example

Without spec: "Create a login form"

With spec:

Login Form Requirements

Fields:

- Email

- Password

- Remember me checkbox

Validation:

- Email must be valid

- Password minimum 8 characters

UI:

- Built using shadcn components

- Tailwind for styling

Behaviour:

- Show validation errors

- Disable submit if invalid

Role of AI in Spec Driven Development

AI becomes extremely powerful when it receives **clear structured instructions**.

Without specs, AI might generate:

- inconsistent code
- wrong architecture
- missing validations
- random styling

With specs, AI becomes a **precise coding assistant**.

AI Role in SDD

AI helps with:Why SDD is Useful

Stage	AI Contribution
specification	refine requirements
plan	suggest architecture
tasks	break work into smaller pieces
implementation	generate structured code
analyze	detect inconsistencies

Using Spec Kit for Feature Development

Spec Kit is a framework created by GitHub to support Spec Driven Development workflows.

It provides CLI tools that help developers:

- generate specs
- structure development
- maintain consistency
- integrate AI effectively

Spec Kit introduces a structured workflow:

/constitution
/specification
/clarify
/plan
/tasks
/analyze
/implement

Each step has a clear purpose.

4. Installing Spec Driven Development Tools

To use Spec Kit, we need the **Python ecosystem CLI runner**.

Tool used - uv / uvx.

Why we use uvx

Important concept - **uvx** is only used to run spec-driven CLI tools.

It is **NOT used for frontend packages**.

Example:

Tool	Purpose
uvx	run spec CLI tools
npm / pnpm	install frontend libraries

Simple Explanation

uvx → runs spec tools (Python)

npm / pnpm → installs frontend libraries (JavaScript)

5. Installing uv

Mac - brew install uv

Windows - winget install --id=astral-sh.uv -e

Once installed, you can run **uvx** to execute **spec** tools.

6. Setting Up an SDD Project

Creating a New Project

uvx --from git+https://github.com/github/spec-kit.git@main specify init **<project-name>**

Example - uvx --from git+https://github.com/github/spec-kit.git@main specify init **login-system**

This will generate the spec driven project structure**.**

Adding SDD to an Existing Project

If your project already exists:

uvx --from git+https://github.com/github/spec-kit.git@main specify init **--here**

This command will add the **spec workflow structure** to your current project.

1. /constitution

Purpose:

Define the project rules and architecture, think of it as the project constitution or law.
It defines things like:

- **tech stack**
- **coding standards**
- **folder structure**
- **UI framework**
- **styling rules**

Example

Tech Stack:

React

TypeScript

TailwindCSS

shadcn UI

Rules:

- use functional components

- use Tailwind for styling

- avoid inline CSS

Why it's important

Without constitution: - every developer writes code differently

With constitution: - codebase stays consistent.

What NOT to do

Do not include:

- business logic
- feature logic
- UI implementation

The Constitution is **global project rules only**.

2. /specification

Purpose:

Define **what features should do**.

Example:
User Login Form
Fields:
- email
- password
- remember me

Validation:
- email required
- password minimum 8 characters

UI:
- use shadcn components

Why it's important

Specs eliminate confusion.

Best Practice

Specs should include:

- UI
- validation
- behaviour
- edge cases

3. /clarify

Purpose:

Resolve missing information in specs.

Example questions:

- Should passwords allow special characters?
- Should I remember to persist in the session?
- What happens on API error?

Clarification ensures **nothing is ambiguous**.

4. /plan

Purpose:

Convert specs into **technical architecture**.

Example plan for login form:

Components

LoginForm.tsx

Libraries

React-hook-form

zod

UI

shadcn input

shadcn button

5. /tasks

Purpose

Break the plan into **small implementation tasks**.

Example:

Task 1

Create LoginForm component

Task 2

Setup form validation

Task 3

Add UI fields

Task 4

Handle form submission

Why tasks matter

Developers can **work independently** without confusion.

6. /analyze

Purpose

Detects issues before coding.

Example problems:

- conflicting requirements
- missing validation
- inconsistent UI

Example:
Spec says password min 8
Plan says password min 6

These kinds of issues are easily caught by Analyze.

7. /implement

Purpose

Generate or write the **actual code**.

Implementation follows:

Constitution

Specification

Plan

Tasks

Example output:

LoginForm.tsx

Using:

- React
- Tailwind
- shadcn
- zod

Best Practices for Frontend Teams

1. Always write specs before coding.
2. Keep specs simple and readable.
3. Use AI only after specs are ready.
4. Follow the constitution strictly.
5. Break work into small tasks.

Common Mistakes

Avoid these mistakes:

Mistake	Problem
writing vague specs	AI generates wrong code
skipping clarify step	missing requirements
ignoring constitution	inconsistent codebase
huge tasks	hard to review

Key Takeaways

Spec Driven Development helps teams:

- reduce confusion
- improve AI usage
- maintain consistent architecture
- build scalable projects

The workflow is simple:

Constitution → Specification → Clarify → Plan → Tasks → Analyze → Implement